

Лабораторная работа №4

Имитационное моделирование

Чистов Даниил Максимович

Содержание

1	Цель работы	6
2	Задание	7
3	Об агентном подходе	8
4	О модели SIR	9
5	SIR в агентном подходе	10
5.1	Пространство состояний	10
5.2	Параметры модели	10
5.3	Динамика	11
6	Выполнение лабораторной работы	12
6.1	Работа с SIR	12
6.2	Код модели SIR	12
6.3	Функция инициализации модели	13
6.4	Шаг агента	16
6.5	Миграция между городами	16
6.6	Передача инфекции	17
6.7	Выздоровление или смерть	18
6.8	Вспомогательные функции для сбора данных	19
6.9	Подключение окружения и зависимостей	20
6.10	Параметры эксперимента	20
6.11	Инициализация модели	21
6.12	Подготовка массивов для хранения данных	21
6.13	Запуск симуляции вручную	21
6.14	Создание DataFrame	22
6.15	Визуализация	22
6.16	Сохранение данных	22
6.17	Подключение окружения и зависимостей	24
6.18	Функция запуска эксперимента	24
6.19	Диапазон параметров	26
6.20	Формирование списка параметров	27
6.21	Запуск экспериментов	27
6.22	Сохранение результатов	28

6.23	Усреднение результатов	28
6.24	Построение графика	28
6.25	Завершение	29
6.26	Подключение окружения и зависимостей	31
6.27	Функция создания матрицы миграции	32
6.28	Функция расчёта времени до пика	32
6.29	Диапазон параметров	34
6.30	Формирование списка параметров	34
6.31	Запуск экспериментов	35
6.32	Сохранение результатов	35
6.33	Усреднение результатов	35
6.34	Визуализация	36
6.35	Завершение	37
6.36	Подключение окружения и зависимостей	38
6.37	Целевая функция оптимизации	38
6.38	Запуск оптимизации	40
6.39	Получение результатов	41
6.40	Вывод результатов	41
6.41	Сохранение результатов	41
6.42	Подключение окружения и зависимостей	42
6.43	Загрузка данных	42
6.44	Построение графиков	42
7	Выводы	45
8	Список литературы	46

Список иллюстраций

6.1	Инициализация проекта	12
6.2	Создание файла <code>sir_run_basic.jl</code>	19
6.3	Результат работы <code>sir_run_basic.jl</code>	23
6.4	Базовый эксперимент SIR	23
6.5	Работа <code>sir_scan_beta.jl</code>	30
6.6	График зависимости коэффициента заразности и эпидимических показателей	30
6.7	Работа <code>sir_migration_effect.jl</code>	37
6.8	График зависимости интенсивности перемещения людей между городами и скорости распространения эпидемии	37
6.9	Результат работы <code>sir_optimize_parameters.jl</code>	41
6.10	Результат работы <code>sir_visualize_dynamics.jl</code>	44

Список таблиц

```
import Pkg
Pkg.activate("../project")
Pkg.instantiate()
```

Activating project at `C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab04\project`

1 Цель работы

Целью данной работы является исследование модели SIR, которая реализована агентным подходом на языке программирования Julia.

2 Задание

- Изучить модель SIR
- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода: чистый код, jupyter notebook, документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Выполнить дополнительные задания

3 Об агентном подходе

Агентный подход к имитационному моделированию (Agent-Based Modeling, ABM) — это метод исследования сложных систем, в котором поведение системы возникает из взаимодействия множества автономных сущностей, называемых агентами. Вместо того чтобы описывать систему глобальными уравнениями, мы моделируем каждую индивидуальную единицу и правила её поведения, а затем наблюдаем, какие коллективные паттерны появляются снизу вверх. Этот подход особенно полезен, когда поведение системы трудно предсказать из-за нелинейностей, гетерогенности участников или адаптивных стратегий.

4 О модели SIR

Модель SIR делит всю популяцию на три взаимосвязанные группы (компарменты), что отражено в её названии:

- S — Susceptible (Восприимчивые): люди, которые не болели, не имеют иммунитета и могут заразиться.
- I — Infectious (Инфицированные/Заразные): люди, которые в данный момент больны и могут передавать инфекцию.
- R — Recovered (Выздоровевшие/Удаленные): люди, которые переболели и приобрели иммунитет (или умерли). Они больше не участвуют в процессе передачи.

Основная цель модели: не предсказать судьбу конкретного человека, а понять общую динамику эпидемии — будет ли она разрастаться, как быстро, сколько людей в итоге переболеет, как влияют карантинные меры.

5 SIR в агентном подходе

Каждый агент — человек, обладающий следующими свойствами:

- `status::Symbol` — состояние (:S, :I, :R);
- `days_infected::Int` — количество дней с момента заражения (для инфицированных).

5.1 Пространство состояний

Агенты размещаются на графе, где узлы представляют локации (города, районы), а рёбра — возможные перемещения между ними. Такой подход позволяет моделировать метапопуляционную динамику и миграцию.

5.2 Параметры модели

- `Ns` — вектор численности населения по локациям;
- `$\beta_{und}$` — вероятность передачи для невыявленных случаев;
- `$\beta_{det}$` — вероятность передачи для выявленных случаев;
- `infection_period` — длительность заболевания (дней);
- `detection_time` — время до выявления заболевания;
- `death_rate` — вероятность летального исхода;
- `reinfection_probability` — вероятность повторного заражения;
- `migration_rates` — матрица вероятностей миграции между локациями.

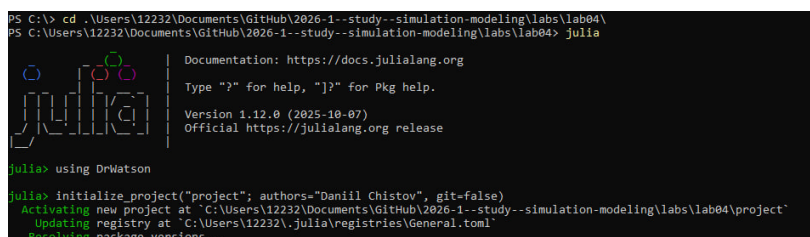
5.3 Динамика

На каждом шаге (соответствует одному дню) для каждого агента выполняются следующие процессы:

- Миграция — агент может переместиться в другую локацию с вероятностью, заданной матрицей `migration_rates`.
- Передача инфекции — если агент инфицирован, он может заразить восприимчивых агентов в той же локации. Количество заражённых за день определяется пуассоновским процессом с интенсивностью, зависящей от статуса выявления (β_{und} или β_{det}).
- Обновление счётчика — для инфицированных увеличивается `days_infected`.
- Выздоровление или смерть — если `days_infected` достиг `infection_period`, агент либо выздоравливает (переходит в R), либо умирает (удаляется из модели) с вероятностью `death_rate`. Выздоровевшие могут быть повторно инфицированы с вероятностью `reinfection_probability`.

6 Выполнение лабораторной работы

В папке lab04 инициализирую проект для данной лабораторной работы (рис. 6.1)



```
PS C:\> cd .\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab04\
PS C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab04> julia

Documentation: https://docs.julialang.org
Type "?" for help, "]?" for Pkg help.
Version 1.12.0 (2025-10-07)
Official https://julialang.org release

julia> using DrWatson
julia> initialize_project("project"; authors="Daniil Chistov", git=false)
Activating new project at "C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab04\project"
Updating registry at "C:\Users\12232\julia\registries\General.toml"
Resolving package versions...
```

Рисунок 6.1: Инициализация проекта

6.1 Работа с SIR

6.1.1 Базовая эксперимент

Предварительно создаю файл sir_model.jl в папке /src. Он будет использоваться последующими программами. Этот код и является моделью SIR. Ниже представлен код sir_model.jl.

6.2 Код модели SIR

```
using Agents, Random
using Agents.Graphs
using StatsBase: sample, Weights
using Distributions: Poisson
```

```

using DrWatson: @dict

# 1000000000 1000 1000000000

@agent struct Person(GraphAgent)
    days_infected::Int # 1000000000 1000 1 1000000000 1000000000
    status::Symbol      # :S, :I, :R
end

```

6.3 Функция инициализации модели

```

function initialize_sir(;
    Ns = [1000, 1000, 1000], # 1000000000 1000 1000000000
    migration_rates = nothing, # 1000000000 1000000000
    β_und = [0.5, 0.5, 0.5], # 1000000000 1000000000000000000
    β_det = [0.05, 0.05, 0.05], # 1000000000 1000000000000000000
    infection_period = 14,
    detection_time = 7,
    death_rate = 0.02,
    reinfection_probability = 0.1,
    Is = [0, 0, 1], # 1000000000 1000000000000 1000000000000000000
    seed = 42,
    n_steps = 100,
)
    rng = Xoshiro(seed)
    C = length(Ns)

    # 10000 1000000000 10000000000 1000 1000000000, 1000000000 1000000000000000000

```

```

if migration_rates == nothing
    migration_rates = zeros(C, C)
    for i = 1:C
        for j = 1:C
            #  $\frac{N_i + N_j}{N_i}$ 
            migration_rates[i, j] = (Ns[i] + Ns[j]) / Ns[i]
        end
    end
    #  $\frac{1}{N_i}$ 
    for i = 1:C
        migration_rates[i, :] ./= sum(migration_rates[i, :])
    end
end

properties = @dict(
    Ns,
     $\lambda_{und}$ ,
     $\lambda_{det}$ ,
    migration_rates,
    infection_period,
    detection_time,
    death_rate,
    reinfection_probability,
    C
)

#  $\frac{1}{N_i} - \frac{1}{N_j}$  (for  $i \neq j$ )
space = GraphSpace(complete_graph(C))

```

```

model = StandardABM(Person, space; properties, rng, (agent_step

# 初始化每个城市的初始状态
for city = 1:C
    for _ = 1:Ns[city]
        add_agent!(city, model, 0, :S)
    end
end

# 初始化每个城市的初始状态
for city = 1:C
    if Is[city] > 0
        # 初始化每个城市的初始状态
        city_agents = ids_in_position(city, model)
        # 初始化每个城市的初始状态
        infected_ids = sample(rng, city_agents, Is[city]; repla
        for id in infected_ids
            agent = model[id]
            agent.status = :I
            agent.days_infected = 1
        end
    end
end

return model
end

```

6.4 Шаг агента

```
function sir_agent_step!(agent, model)
    # Обновляем координаты
    migrate!(agent, model)

    # Обновляем статус (если агент заражен)
    if agent.status == :I
        transmit!(agent, model)
    end

    # Обновляем количество дней болезни
    if agent.status == :I
        agent.days_infected += 1
    end

    # Обновляем статус (если агент выздоровел или умер)
    recover_or_die!(agent, model)
end
```

6.5 Миграция между городами

```
function migrate!(agent, model)
    current_city = agent.pos
    # Выбираем целевой город
    probs = model.migration_rates[current_city, :]
    target = sample(abmrng(model), 1:model.C, Weights(probs))
```



```

if target != current_city
    move_agent!(agent, target, model)
end
end

```

6.6 Передача инфекции

```

function transmit!(agent, model)
    # Сколько вероятно передаст инфекцию в данный момент
    rate = if agent.days_infected < model.detection_time
        model._und[agent.pos] # Вероятность передачи инфекции
    else
        model._det[agent.pos] # Вероятность передачи инфекции
    end

    # Сколько инфекций передаст в данный момент
    n_infections = rand(abmrng(model), Poisson(rate))
    n_infections == 0 && return nothing

    # Сколько соседей есть в данный момент, среди которых есть инфицированные
    neighbors = [a for a in agents_in_position(agent.pos, model) if

    # Случайно выбираем соседей
    shuffle!(abmrng(model), neighbors)

    for contact in neighbors
        # Сколько вероятно передаст инфекцию в данный момент
        if contact.status == :S

```

```

        contact.status = :I
        contact.days_infected = 1
        n_infections -= 1
        n_infections == 0 && return nothing
    elseif contact.status == :R && rand(abmrng(model)) < model.r
        contact.status = :I
        contact.days_infected = 1
        n_infections -= 1
        n_infections == 0 && return nothing
    end
end
end
end

```

6.7 Выздоровление или смерть

```

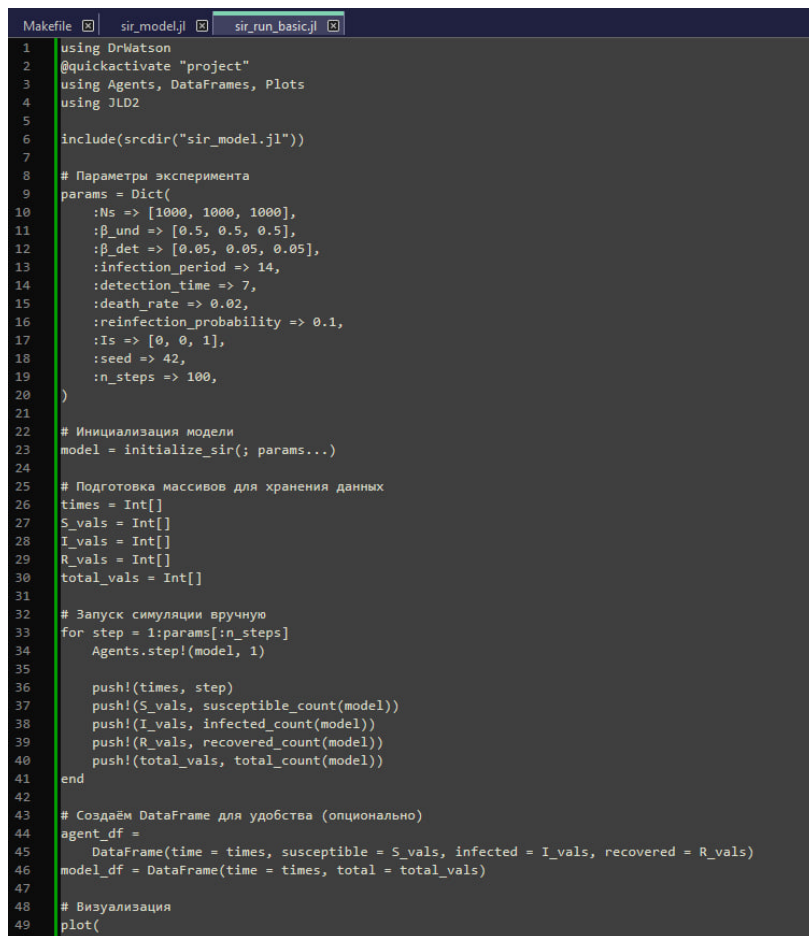
function recover_or_die!(agent, model)
    if agent.status == :I && agent.days_infected < model.infection_p
        if rand(abmrng(model)) < model.death_rate
            remove_agent!(agent, model) # XXXXX (XXXXXXXXXX)
        else
            agent.status = :R # XXXXXXXXXXXXXXXX
            agent.days_infected = 0
        end
    end
end
end
end

```

6.8 Вспомогательные функции для сбора данных

```
infected_count(model) = count(a.status == :I for a in allagents(model))
recovered_count(model) = count(a.status == :R for a in allagents(model))
susceptible_count(model) = count(a.status == :S for a in allagents(model))
total_count(model) = nagents(model)
```

Создаю файл `sir_run_basic.jl` в папке `/scripts`. Это базовый эксперимент (рис. 6.2).



```
1 using DrWatson
2 @quickactivate "project"
3 using Agents, DataFrames, Plots
4 using JLD2
5
6 include(sourcedir("sir_model.jl"))
7
8 # Параметры эксперимента
9 params = Dict{
10     :Ns => [1000, 1000, 1000],
11     :β_und => [0.5, 0.5, 0.5],
12     :β_det => [0.05, 0.05, 0.05],
13     :infection_period => 14,
14     :detection_time => 7,
15     :death_rate => 0.02,
16     :reinfection_probability => 0.1,
17     :Is => [0, 0, 1],
18     :seed => 42,
19     :n_steps => 100,
20 }
21
22 # Инициализация модели
23 model = initialize_sir(; params...)
24
25 # Подготовка массивов для хранения данных
26 times = Int[]
27 S_vals = Int[]
28 I_vals = Int[]
29 R_vals = Int[]
30 total_vals = Int[]
31
32 # Запуск симуляции вручную
33 for step = 1:params[:n_steps]
34     Agents.step!(model, 1)
35
36     push!(times, step)
37     push!(S_vals, susceptible_count(model))
38     push!(I_vals, infected_count(model))
39     push!(R_vals, recovered_count(model))
40     push!(total_vals, total_count(model))
41 end
42
43 # Создаём DataFrame для удобства (опционально)
44 agent_df =
45     DataFrame(time = times, susceptible = S_vals, infected = I_vals, recovered = R_vals)
46 model_df = DataFrame(time = times, total = total_vals)
47
48 # Визуализация
49 plot(
```

Рисунок 6.2: Создание файла `sir_run_basic.jl`

6.9 Подключение окружения и зависимостей

```
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots
using JLD2

include(sourcedir("sir_model.jl"))
```

6.10 Параметры эксперимента

```
params = Dict(
    :Ns => [1000, 1000, 1000],
    :X_und => [0.5, 0.5, 0.5],
    :X_det => [0.05, 0.05, 0.05],
    :infection_period => 14,
    :detection_time => 7,
    :death_rate => 0.02,
    :reinfection_probability => 0.1,
    :Is => [0, 0, 1],
    :seed => 42,
    :n_steps => 100,
)
```

6.11 Инициализация модели

```
model = initialize_sir(; params...)
```

6.12 Подготовка массивов для хранения данных

```
times = Int[]  
S_vals = Int[]  
I_vals = Int[]  
R_vals = Int[]  
total_vals = Int[]
```

6.13 Запуск симуляции вручную

```
for step = 1:params[:n_steps]  
    Agents.step!(model, 1)  
  
    push!(times, step)  
    push!(S_vals, susceptible_count(model))  
    push!(I_vals, infected_count(model))  
    push!(R_vals, recovered_count(model))  
    push!(total_vals, total_count(model))  
end
```

6.14 Создание DataFrame

```
agent_df =  
    DataFrame(time = times, susceptible = S_vals, infected = I_vals)  
  
model_df = DataFrame(time = times, total = total_vals)
```

6.15 Визуализация

```
plot(  
    agent_df.time,  
    agent_df.susceptible,  
    label = "S",  
    xlabel = "time",  
    ylabel = "S",  
)  
plot!(agent_df.time, agent_df.infected, label = "I")  
plot!(agent_df.time, agent_df.recovered, label = "R")  
plot!(agent_df.time, model_df.total, label = "Total (S + I + R)",  
savefig(plotsdir("sir_basic_dynamics.png"))
```

6.16 Сохранение данных

```
@save datadir("sir_basic_agent.jld2") agent_df  
@save datadir("sir_basic_model.jld2") model_df
```

Код успешно работает - результаты расположены в соответствующих папках (рис. 6.3).

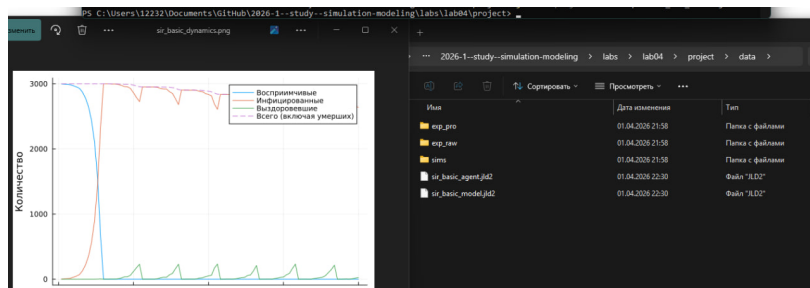


Рисунок 6.3: Результат работы sir_run_basic.jl

Ниже представлен график базового эксперимента (рис. 6.4).

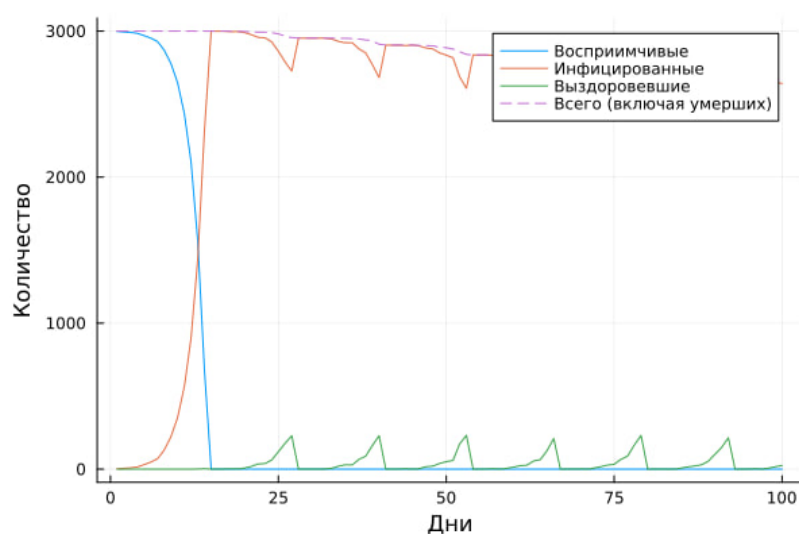


Рисунок 6.4: Базовый эксперимент SIR

6.16.1 Дополнительное задание №1

Требуется определить базовое репродуктивное число R_0 по формуле $R_0 = \beta/\gamma$, где $\gamma = 1/\text{infection_period}$. Затем сравнить с наблюдаемой динамикой.

Требуемые для решения задачи параметры можно найти в коде:

- $\beta_{\text{und}} \Rightarrow [0.5, 0.5, 0.5]$, т. е. всегда 0.5
- $\text{infection_period} \Rightarrow 14$

Тогда $R_0 = (0.5) * 14 = 7$

Так как базовое репродуктивное число значительно больше 1, следует ожидать быстрой рост. Это соответствует полученному результату на графике

6.16.2 Сканирование коэффициента заразности

По заданию требуется создать код `sir_scan_beta.jl`.

Данный код исследует, как изменение базовой заразности (β_{und} и пропорционально β_{det}) влияет на эпидемические показатели: пик заболеваемости, долю переболевших, число умерших. Выполняется параметрическое сканирование с несколькими повторными прогонами для учёта стохастичности.

Ниже представлен код `sir_scan_beta.jl`

6.17 Подключение окружения и зависимостей

```
using DrWatson
@quickactivate "project"

using Agents, DataFrames, Plots, CSV, Random

include(sourcedir("sir_model.jl"))
```

6.18 Функция запуска эксперимента

```
function run_experiment(p)
    beta = p[:beta]
     $\beta_{und}$  = fill(beta, 3)
     $\beta_{det}$  = fill(beta/10, 3)
```



```

model = initialize_sir(;
    Ns = p[:Ns],
    ⍺_und = ⍺_und,
    ⍺_det = ⍺_det,
    infection_period = p[:infection_period],
    detection_time = p[:detection_time],
    death_rate = p[:death_rate],
    reinfection_probability = p[:reinfection_probability],
    Is = p[:Is],
    seed = p[:seed],
    n_steps = p[:n_steps],
)

infected_fraction(model) =
    count(a.status == :I for a in allagents(model)) / nagents(model)

peak_infected = 0.0

for step = 1:p[:n_steps]
    agent_ids = collect(allids(model))
    for id in agent_ids
        agent = try
            model[id]
        catch
            nothing
        end
        if agent != nothing
            sir_agent_step!(agent, model)
        end
    end
end

```

```

        end
    end

    frac = infected_fraction(model)
    if frac > peak_infected
        peak_infected = frac
    end
end

final_infected = infected_fraction(model)
final_recovered = count(a.status == :R for a in allagents(model)
total_deaths = sum(p[:Ns]) - nagents(model)

return (
    peak = peak_infected,
    final_inf = final_infected,
    final_rec = final_recovered,
    deaths = total_deaths,
)
end

```

6.19 Диапазон параметров

```

beta_range = 0.1:0.1:1.0
seeds = [42, 43, 44]

```

6.20 Формирование списка параметров

```
params_list = []

for b in beta_range
  for s in seeds
    push!(
      params_list,
      Dict(
        :beta => b,
        :Ns => [1000, 1000, 1000],
        :infection_period => 14,
        :detection_time => 7,
        :death_rate => 0.02,
        :reinfection_probability => 0.1,
        :Is => [0, 0, 1],
        :seed => s,
        :n_steps => 100,
      ),
    )
  end
end
```

6.21 Запуск экспериментов

```
results = []

for params in params_list
```

```

data = run_experiment(params)
push!(results, merge(params, Dict(pairs(data))))
println("XXXXXXXX XXXXXXXXXXXX β beta = $(params[:beta]), seed = $(pa
end

```

6.22 Сохранение результатов

```

df = DataFrame(results)
CSV.write(datadir("beta_scan_all.csv"), df)

```

6.23 Усреднение результатов

```

using Statistics

grouped = combine(
    groupby(df, [:beta]),
    :peak => mean => :mean_peak,
    :final_inf => mean => :mean_final_inf,
    :deaths => mean => :mean_deaths,
)

```

6.24 Построение графика

```

plot(
    grouped.beta,
    grouped.mean_peak,

```

```

        label = "R0 [95% CI]",
        xlabel = "R0 [95% CI] (mean)",
        ylabel = "R0 [95% CI] (mean)",
        marker = :circle,
        linewidth = 2,
    )

    plot!(
        grouped.beta,
        grouped.mean_final_inf,
        label = "R0 [95% CI] (mean)",
        marker = :square,
    )

    plot!(
        grouped.beta,
        grouped.mean_deaths ./ 3000,
        label = "R0 [95% CI]",
        marker = :diamond,
    )

    savefig(plotsdir("beta_scan.png"))

```

6.25 Завершение

```
println("R0 [95% CI] (mean) data/beta_scan_all.csv plots/beta_scan")
```

Код успешно работает (рис. 6.5).

```

Завершён эксперимент с beta = 0.1, seed = 43
Завершён эксперимент с beta = 0.1, seed = 44
Завершён эксперимент с beta = 0.2, seed = 42
Завершён эксперимент с beta = 0.2, seed = 43
Завершён эксперимент с beta = 0.2, seed = 44
Завершён эксперимент с beta = 0.3, seed = 42
Завершён эксперимент с beta = 0.3, seed = 43
Завершён эксперимент с beta = 0.3, seed = 44
Завершён эксперимент с beta = 0.4, seed = 42
Завершён эксперимент с beta = 0.4, seed = 43
Завершён эксперимент с beta = 0.4, seed = 44
Завершён эксперимент с beta = 0.5, seed = 42
Завершён эксперимент с beta = 0.5, seed = 43
Завершён эксперимент с beta = 0.5, seed = 44
Завершён эксперимент с beta = 0.6, seed = 42
Завершён эксперимент с beta = 0.6, seed = 43
Завершён эксперимент с beta = 0.6, seed = 44
Завершён эксперимент с beta = 0.7, seed = 42
Завершён эксперимент с beta = 0.7, seed = 43
Завершён эксперимент с beta = 0.7, seed = 44
Завершён эксперимент с beta = 0.8, seed = 42
Завершён эксперимент с beta = 0.8, seed = 43
Завершён эксперимент с beta = 0.8, seed = 44
Завершён эксперимент с beta = 0.9, seed = 42
Завершён эксперимент с beta = 0.9, seed = 43
Завершён эксперимент с beta = 0.9, seed = 44
Завершён эксперимент с beta = 1.0, seed = 42
Завершён эксперимент с beta = 1.0, seed = 43
Завершён эксперимент с beta = 1.0, seed = 44
Результаты сохранены в data/beta_scan_all.csv и plots/beta_scan.png

```

Рисунок 6.5: Работа sir_scan_beta.jl

Ниже представлен график, полученный в результате работы кода (рис. 6.6).

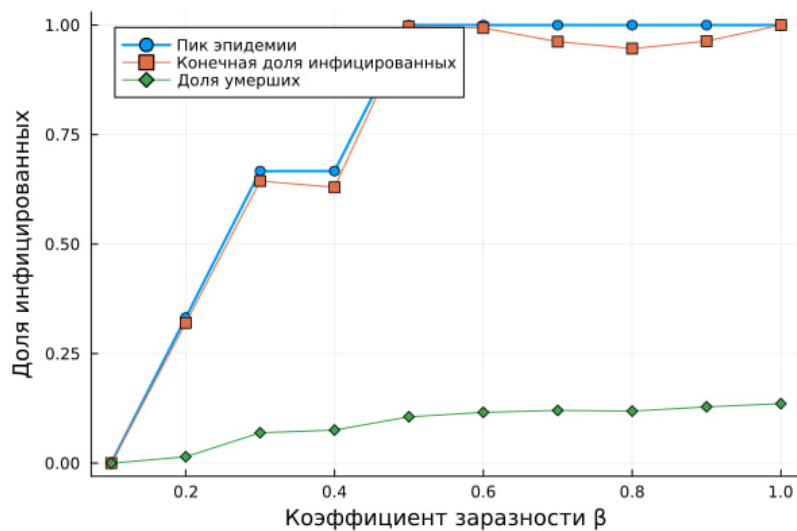


Рисунок 6.6: График зависимости коэффициента заразности и эпидимических показателей

6.25.1 Дополнительное задание №2

Требуется:

- Найти минимальное значение β , при котором возникает эпидемия (пик $I > 5\%$ популяции).
- Сравнить с теоретическим порогом $R_0 = 1$.

Из кода `sir_scan_beta.jl` известен размер популяции: `Ns => [1000, 1000, 1000]`. Следовательно вся популяция вместе $N = 3000$, а значит нужно найти такое значение β , при котором возникает эпидемия (пик $I > 150$ популяции).

По графику можно сделать вывод, что эпидемия возникнет при β значительно меньше 0.2.

6.25.2 Исследование эффекта миграции

Требуется создать код, который исследует, как интенсивность перемещения людей между городами влияет на скорость распространения эпидемии (время достижения пика) и масштаб пика. Инфекция начинается только в одном городе, остальные изначально здоровы.

Ниже представлена реализация данного кода: `sir_migration_effect.jl`

6.26 Подключение окружения и зависимостей

```
using DrWatson
@quickactivate "project"

using Agents, DataFrames, Plots, CSV, Random

include(sourcedir("sir_model.jl"))
```

6.27 Функция создания матрицы миграции

```
function create_migration_matrix(C, intensity)
    M = ones(C, C) .* intensity ./ (C-1)
    for i = 1:C
        M[i, i] = 1 - intensity
    end
    return M
end
```

6.28 Функция расчёта времени до пика

```
function peak_time(p)
    migration_rates = create_migration_matrix(p[:C], p[:migration_i

    model = initialize_sir(;
        Ns = p[:Ns],
        ⌀_und = p[:⌀_und],
        ⌀_det = p[:⌀_det],
        infection_period = p[:infection_period],
        detection_time = p[:detection_time],
        death_rate = p[:death_rate],
        reinfection_probability = p[:reinfection_probability],
        Is = p[:Is],
        seed = p[:seed],
        migration_rates = migration_rates,
    )
```



```

    infected_frac(model) = count(a.status == :I for a in allagents(model))

    peak = 0.0
    peak_step = 0

    for step = 1:p[:n_steps]
        agent_ids = collect(allids(model))
        for id in agent_ids
            agent = try
                model[id]
            catch
                nothing
            end
            if agent != nothing
                sir_agent_step!(agent, model)
            end
        end

        frac = infected_frac(model)
        if frac > peak
            peak = frac
            peak_step = step
        end
    end

    return (peak_time = peak_step, peak_value = peak)
end

```

6.29 Диапазон параметров

```
migration_intensities = 0.0:0.1:0.5  
seeds = [42, 43, 44]
```

6.30 Формирование списка параметров

```
params_list = []  
  
for mig in migration_intensities  
    for s in seeds  
        push!(  
            params_list,  
            Dict(  
                :migration_intensity => mig,  
                :C => 3,  
                :Ns => [1000, 1000, 1000],  
                :X_und => [0.5, 0.5, 0.5],  
                :X_det => [0.05, 0.05, 0.05],  
                :infection_period => 14,  
                :detection_time => 7,  
                :death_rate => 0.02,  
                :reinfection_probability => 0.1,  
                :Is => [1, 0, 0],  
                :seed => s,  
                :n_steps => 150,  
            ),  
        )
```

```
end  
end
```

6.31 Запуск экспериментов

```
results = []  
  
for params in params_list  
    data = peak_time(params)  
    push!(results, merge(params, Dict{pairs(data)}))  
    println(  
        "XXXXXXXXX XXXXXXXXXXXXXXX migration_intensity = $(params[:migration_intensity])  
    )  
end
```

6.32 Сохранение результатов

```
df = DataFrame(results)  
CSV.write(datadir("migration_scan_all.csv"), df)
```

6.33 Усреднение результатов

```
using Statistics  
  
grouped = combine(  
    groupby(df, [:migration_intensity]),
```

```

:peak_time => mean => :mean_peak_time,
:peak_value => mean => :mean_peak_value,
)

```

6.34 Визуализация

```

plot(
  grouped.migration_intensity,
  grouped.mean_peak_time,
  marker = :circle,
  xlabel = "XXXXXXXXXXXXXXXX XXXXXXXX",
  ylabel = "XXXXX XX XXXX (XXX)",
  label = "XXXXX XXXX",
)

plot!(
  grouped.migration_intensity,
  grouped.mean_peak_value .* 3000,
  marker = :square,
  xlabel = "XXXXXXXXXXXXXXXX XXXXXXXX",
  ylabel = "XXXXXXXXXXXXX X XXXX",
  label = "XXXXXX XXXXXXXXXXXXXXXX",
)

savefig(plotsdir("migration_effect.png"))

```

6.35 Завершение

```
println("XXXXXXXXXX XXXXXXXXXX X data/migration_scan_all.csv X plots/migr
```

Код успешно работает (рис. 6.7).

```
PS C:\Users\12232\Documents\GitHub\2020-1--study--simulation-modeling\labs\lab04\project> julia --project=. .\scripts\sir_migration_effect.jl
Завершен эксперимент с migration_intensity = 0.0, seed = 42
Завершен эксперимент с migration_intensity = 0.0, seed = 43
Завершен эксперимент с migration_intensity = 0.0, seed = 44
Завершен эксперимент с migration_intensity = 0.1, seed = 42
Завершен эксперимент с migration_intensity = 0.1, seed = 43
Завершен эксперимент с migration_intensity = 0.1, seed = 44
Завершен эксперимент с migration_intensity = 0.2, seed = 42
Завершен эксперимент с migration_intensity = 0.2, seed = 43
Завершен эксперимент с migration_intensity = 0.2, seed = 44
Завершен эксперимент с migration_intensity = 0.3, seed = 42
Завершен эксперимент с migration_intensity = 0.3, seed = 43
Завершен эксперимент с migration_intensity = 0.3, seed = 44
Завершен эксперимент с migration_intensity = 0.4, seed = 42
Завершен эксперимент с migration_intensity = 0.4, seed = 43
Завершен эксперимент с migration_intensity = 0.4, seed = 44
Завершен эксперимент с migration_intensity = 0.5, seed = 42
Завершен эксперимент с migration_intensity = 0.5, seed = 43
Завершен эксперимент с migration_intensity = 0.5, seed = 44
результаты сохранены в data/migration_scan_all.csv в plots/migration_effect.png
```

Рисунок 6.7: Работа sir_migration_effect.jl

Ниже представлен график, полученный в результате работы кода (рис. 6.8).

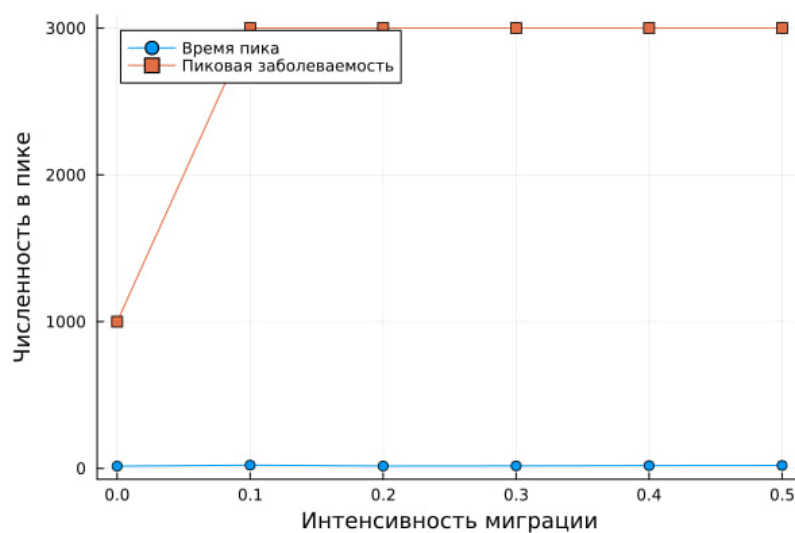


Рисунок 6.8: График зависимости интенсивности перемещения людей между городами и скорости распространения эпидемии

6.35.1 Многокритериальная оптимизация параметров

По заданию был создан код, который ищет оптимальные комбинации параметров, минимизирующие одновременно два критерия: пиковую заболеваемость и долю умерших.

Ниже его реализация: sir_optimize_parameters.jl

6.36 Подключение окружения и зависимостей

```
using DrWatson
@quickactivate "project"

using BlackBoxOptim, Random, Statistics

include(sourcedir("sir_model.jl"))
```

6.37 Целевая функция оптимизации

```
function cost_multi(x)
    model = initialize_sir(;
        Ns = [1000, 1000, 1000],
        ⍺_und = fill(x[1], 3),
        ⍺_det = fill(x[1]/10, 3),
        infection_period = 14,
        detection_time = round{Int}(x[2]),
        death_rate = x[3],
        reinfection_probability = 0.1,
        Is = [0, 0, 1],
        seed = 42,
        n_steps = 100,
    )
```

```

infected_frac(model) = count(a.status == :I for a in allagents(model))
dead_count(model) = 3000 - nagents(model)

peak_infected = 0.0

replicates = 5
peak_vals = Float64[]
dead_vals = Int[]

for rep = 1:replicates
    model = initialize_sir(;
        Ns = [1000, 1000, 1000],
        x_und = fill(x[1], 3),
        x_det = fill(x[1]/10, 3),
        infection_period = 14,
        detection_time = round{Int}(x[2]),
        death_rate = x[3],
        reinfection_probability = 0.1,
        Is = [0, 0, 1],
        seed = 42 + rep,
        n_steps = 100,
    )

    for step = 1:100
        Agents.step!(model, 1)
        frac = infected_frac(model)
        if frac > peak_infected
            peak_infected = frac
        end
    end
end

```

```

        end

    end

    push!(peak_vals, peak_infected)
    push!(dead_vals, dead_count(model))
end

return (mean(peak_vals), mean(dead_vals) / 3000)
end

```

6.38 Запуск оптимизации

```

result = bboptimize(
    cost_multi,
    Method = :borg_moea,
    FitnessScheme = ParetoFitnessScheme{2}(is_minimizing = true),
    SearchRange = [
        (0.1, 1.0),
        (3.0, 14.0),
        (0.01, 0.1),
    ],
    NumDimensions = 3,
    MaxTime = 120,
    TraceMode = :compact,
)

```


6.39 Получение результатов

```
best = best_candidate(result)
fitness = best_fitness(result)
```

6.40 Вывод результатов

```
println("XXXXXXXXXXXX XXXXXXXXXX:")
println("x_und = $(best[1])")
println("XXXXXX XXXXXXXXXX = $(round{Int, best[2]}) XXXX")
println("XXXXXXXXXXXX = $(best[3])")

println("XXXXXXXXXXXX XXXXXXXXXX:")
println("XXX XXXXXXXXXXXXXXX: $(fitness[1])")
println("XXXX XXXXXXX: $(fitness[2])")
```

6.41 Сохранение результатов

```
save(datadir("optimization_result.jld2"), Dict("best" => best, "fit"
```

Код успешно работает и приводит следующие результаты в терминале (рис. 6.9).

```
PS C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab04\project> julia --project=. \scripts\sir_optimize_parameters.jl
Starting optimization with optimizer BlackBoxOptim.BorgMOEA(EpsBoxDominanceFitnessScheme{2, Float64, true, typeof(sum)}, BlackBoxOptim.ProblemEvaluator(Tuple{Float64, Float64}, IndexedTupleFitness{2, Float64}, EpsBoxArchive{2, Float64, EpsBoxDominanceFitnessScheme{2, Float64, true, typeof(sum)}}, FunctionBase{Problem{typeof(cost_multi), ParetoFitnessScheme{2, Float64, true, typeof(sum)}, ContinuousRectSearchSpace, Nothing}}, FitPopulation{IndexedTupleFitness{2, Float64}}, FixedGeneticOperatorsMixture, RandomBound{ContinuousRectSearchSpace}), 0.00 secs, 0 evals, 0 steps
pop_size=50 arch_size=0 n_restarts=0

Optimization stopped after 1 steps and 1225.95 seconds
Termination reason: Max time (120.0 s) reached
Steps per second = 0.00
Function evals per second = 0.04
Improvements/step = NaN
Total function evaluations = 51

Best candidate found: [0.107901, 8.99207, 0.0919948]
Fitness: (0.00167, 0.00020) agg=0.00187

Оптимизационные параметры:
x_und = 0.10790145420672019
Время вычисления = 9 дней
Улучшенность = 0.00199482615076235
Достижимые показатели:
Мак. заболеваемость: 0.001668335001668335
Поп. умерших: 0.00019999999999999998
PS C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab04\project>
```

Рисунок 6.9: Результат работы sir_optimize_parameters.jl

6.41.1 Сводная визуализация результатов

Для выполнения работы также требуется создать код, который загружает результаты параметрического сканирования (файл `data/beta_scan_all.csv`, созданный `scan_beta.jl`) и строит единый составной график, объединяющий три панели:

- пик эпидемии и конечная доля инфицированных;
- число умерших;
- доля выздоровевших.

Ниже его реализация: `sir_visualize_dynamic.jl`

6.42 Подключение окружения и зависимостей

```
using DrWatson
@quickactivate "project"

using Agents, DataFrames, Plots, CSV

include(scrdir("sir_model.jl"))
```

6.43 Загрузка данных

```
df = CSV.read(datadir("beta_scan_all.csv"), DataFrame)
```

6.44 Построение графиков

```

p1 = plot(df.beta, df.peak, label = "peak", xlabel = "beta", ylabel = "peak")
plot!(p1, df.beta, df.final_inf, label = "final_inf")

p2 = plot(df.beta, df.deaths, xlabel = "beta", ylabel = "deaths")

p3 = plot(df.beta, df.final_rec, xlabel = "beta", ylabel = "final_rec")

plot(p1, p2, p3, layout = (3, 1), size = (800, 900))

savefig(plotsdir("comprehensive_analysis.png"))

```

Код успешно работает. В результате был получен следующий график (рис. 6.10).

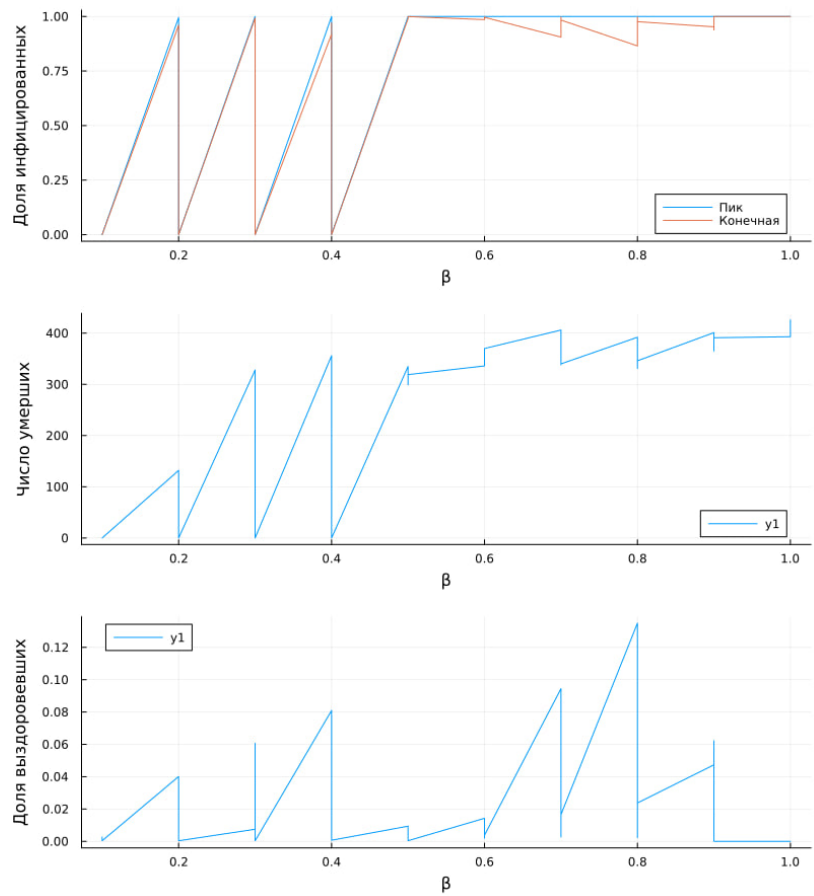


Рисунок 6.10: Результат работы sir_visualize_dynamics.jl

7 Выводы

В результате выполнения данной лабораторной работы я закрепил навыки работы с агентным подходом имитационного моделирования на примере модели SIR, реализованной на Julia. Разобрался в самой модели и интегрировал её реализацию в данный отчёт.

8 Список литературы

Лабораторная работа №4